

爬虫 + 数据处理分析 + Agent 数据支持

面试复习与项目深挖手册

针对岗位 JD：外部数据采集、Python 爬虫、pandas/SQL/Excel 分析、Agent 数据支持

使用方法：先按“知识点地图”补基础，再按“项目深挖”把每个知识点绑定到自己的项目，最后按“高频问答模板”练 1-2 分钟回答。目标不是背名词，而是在面试官追问时能把业务、技术、数据和项目证据串起来。

目录

- 一、岗位能力模型：面试官到底在考什么
- 二、第一大块：爬虫采集基础
- 三、第二大块：数据处理与分析基础
- 四、第三大块：Agent / LLM 相关基础
- 五、结合你的项目：项目深挖与回答口径
- 六、知识点 + 项目绑定表
- 七、高频面试题回答模板
- 八、代码模板与排查清单
- 九、14 天冲刺复习计划
- 十、模拟面试脚本

一、岗位能力模型：面试官到底在考什么

这个岗位不是单纯“爬虫工程师”，也不是单纯“数据分析师”。它的真实定位是：能够根据业务需求拆解外部数据源，使用 Python 爬虫稳定采集网页/API/平台数据，再用 pandas/SQL/Excel 清洗和分析，并把流程沉淀给 Agent 或业务使用。

一句话人设：我能从业务问题出发，找到数据源，分析网页/API，写 Python 脚本采集，处理分页、动态渲染、重试和限速，再把数据清洗成可分析的结构化表，输出分析结论，并理解这些数据如何服务 Agent/RAG。

1.1 岗位能力拆解

能力层	JD 对应要求	你要证明什么	常见追问
数据源拆解	采集外包平台、公开网页、API、榜单、论坛	能判断数据在哪里、字段怎么设计	给你一个网站怎么开始？
爬虫采集	Python 爬虫脚本、分页、动态渲染、接口数据	能稳定拿到数据	requests/Selenium/Scrapy 怎么选？
稳定性工程	异常重试、限速	脚本不是跑一次，而是可维护	403/429/timeout 怎么处理？
数据清洗	去重、字段抽取、格式转换、质量检查	数据能用、能分析	JSON 嵌套、缺失、重复怎么办？
分析交付	pandas / SQL / Excel 输出结构化表和结论	能把结果给业务方	如何统计、分组、输出 Excel？
Agent 支持	拆数据源、字段结构、采集方案，LLM/Agent/RAG	理解数据如何进入 Agent 流程	Agent 为什么需要结构化数据？

1.2 你面试时应该反复强调的完整链路

- 业务需求
- > 明确字段：标题、预算、发布时间、标签、发布方、描述等
 - > 分析数据源：HTML / XHR API / 动态渲染 / App 接口
 - > 选择工具：requests / Scrapy / Selenium / Playwright / DrissionPage
 - > 处理采集问题：分页、headers、Cookie、timeout、retry、限速
 - > 解析响应：HTML、JSON、图片、文本、二级评论
 - > 数据清洗：去重、缺失值、异常值、字段标准化、时间格式
 - > 分析输出：CSV / Excel / SQL 表、pandas 分析、Markdown 结论
 - > 流程沉淀：字段字典、采集方案、失败任务重跑、Agent 可复用

1.3 开场自我介绍的技术版

我主要做数据采集与数据处理分析。采集侧，我接触过公开网页、接口 JSON、动态页面和平台数据，熟悉 requests、Scrapy、Selenium/DrissionPage 等工具，能处理分页、请求头、Cookie、异常重试和限速。数据侧，我做过金融文本清洗、财务风险预测、复杂 JSON 解析和 pandas/SQL 分析，重视字段一致性、重复值、异常值和输出可复现。Agent 侧，我做过 CSV 数据分析 Agent，理解结构化数据如何服务 LLM/Agent 场景。

二、第一大块：爬虫采集基础

2.1 爬虫任务完整流程

面试官最容易用开放题考你：“给你一个外包平台/招聘网站/榜单/论坛，你怎么采？”你必须先把流程讲清楚，而不是直接说“我用 requests”。

你要掌握的流程

- 明确业务目标：为什么采、采给谁用、最终要分析什么。
- 定义字段：字段名、字段类型、来源页面、是否必填、是否唯一。
- 分析页面：静态 HTML、XHR/API、动态渲染、登录态、分页。
- 选择工具：requests 处理接口，BeautifulSoup/XPath 处理 HTML，Selenium/Playwright/DrissionPage 处理浏览器场景，Scrapy 处理工程化。
- 稳定采集：timeout、retry、限速、日志、失败任务记录、断点续爬。
- 解析保存：HTML/JSON 解析，写 CSV/Excel/数据库。
- 清洗质检：重复、缺失、异常、类型、时间、枚举、字段一致性。
- 交付沉淀：结构化表、分析结论、字段字典、Markdown 流程。

标准回答模板

我会先确定业务字段，再用开发者工具判断数据来自 HTML、XHR/API 还是动态渲染。接口能复现就优先用 requests/httpx；需要点击、滚动、登录或 JS 执行时用 Selenium/Playwright/DrissionPage；长期结构化项目用 Scrapy。采集时处理分页、headers、Cookie、timeout、retry 和限速；采集后用 pandas 做清洗和质量检查，最终输出 CSV/Excel 和流程文档。

2.2 HTTP / HTTPS / URL 基础

HTTP 是爬虫的地基。你不需要一开始背 TLS 的每一步，但必须知道请求和响应的基本结构。

URL 结构

https://www.example.com:443/search/list?page=2&keyword=python#section1
协议 域名 端口 路径 查询参数 锚点

- 协议：http 或 https。https 表示在 HTTP 基础上使用 TLS 加密。
- 域名：目标服务器地址，例如 www.example.com。
- 路径：服务器资源位置，例如 /api/list。
- 查询参数：?page=2&keyword=python，通常用于搜索、分页、筛选。
- 锚点：#section1，通常只在浏览器端定位页面位置，不会发给服务器。

HTTP 请求流程

输入 URL

- > DNS 把域名解析成 IP
- > 建立 TCP 连接
- > 如果是 HTTPS，进行 TLS 加密协商
- > 发送 HTTP 请求：method + path + headers + body
- > 服务器处理请求
- > 返回 HTTP 响应：status code + headers + body
- > 客户端解析 HTML / JSON / 图片 / 文件

GET 和 POST

对比项	GET	POST
常见用途	查询、搜索、分页、获取资源	提交表单、提交 JSON、复杂查询
参数位置	URL 查询参数 params	请求体 body：data/json/form-data
爬虫复现重点	看 Query Params	看 Payload 和 Content-Type
是否有请求体	通常没有	通常有

面试回答：GET 和 POST 区别

GET 通常用于获取资源，参数多在 URL 查询字符串里；POST 通常用于提交数据，参数在请求体里，可以是表单或 JSON。爬虫复现接口时，我会看请求方法、Query Params、Payload 和 Content-Type 来决定用 requests.get 的 params，还是 requests.post 的 data/json。

2.3 请求头 headers 与响应头

很多接口失败，不是 URL 错，而是 headers、Cookie、Referer、Origin、Content-Type 等没有复现完整。

字段	含义	面试重点
User-Agent	客户端身份，例如 Chrome、移动端浏览器	常用于基础请求伪装，过于异常会被识别
Cookie	浏览器保存的会话信息	登录态、session_id、偏好设置，常决定能否访问
Referer	请求来源页面	部分网站会检查是否从合法页面跳转
Origin	请求源站点	常用于跨域请求校验，尤其 POST
Accept	客户端希望接收的数据类型	HTML/JSON/图片等内容协商
Accept-Language	语言偏好	部分站点返回中英文内容不同
Content-Type	请求体格式	application/json 或 form-urlencoded
Authorization	认证信息，如 Bearer token	涉及权限时必须合法合规处理

面试回答：headers 为什么重要

headers 体现浏览器环境、来源、登录态和请求体格式。分析接口时我会对比浏览器请求和 Python 请求，重点看 User-Agent、Cookie、Referer、Origin、Content-Type。如果 403 或返回空数据，通常先检查这些字段。

2.4 Cookie、Session、Token

Cookie 是客户端保存的小段数据；Session 通常是服务端保存的会话状态；Token 是某种认证令牌。爬虫里常见问题是：浏览器能访问，Python 请求不行，原因可能就是缺少 Cookie、Token 或 Session 状态。

概念	存放位置	作用	爬虫里的典型问题
Cookie	浏览器/客户端	保存 session_id、偏好、登录状态	没有 Cookie，接口返回未登录或 403
Session	服务端为主，客户端用 Cookie 存 session_id	维持一次登录后的状态	请求之间不连续，状态丢失
Token	请求头、Cookie、LocalStorage 等	认证和授权	Token 过期或缺失导致 401/403

面试回答：Cookie 和 Session 区别

Cookie 存在客户端，Session 通常存在服务端。服务端通过 Cookie 里的 session_id 找到对应 Session。爬虫里如果需要登录态，我会使用 requests.Session 或浏览器自动化先建立会话，再复用 Cookie 或合法的会话状态。

2.5 状态码与排查

状态码	含义	应对思路
200	成功	继续解析 response.text/json/content
301/302	重定向	检查 Location，是否需要登录或跳转
400	请求参数错误	检查 params/payload/Content-Type
401	未认证	检查登录、Cookie、Token
403	拒绝访问	检查权限、headers、Cookie、Referer、频率、动态参数
404	资源不存在	检查 URL 是否过期或参数错误
429	请求过快	降速、退避重试、降低并发
500	服务器错误	可重试，也要检查参数是否导致服务异常
502/503/504	网关/服务暂不可用	重试、记录失败、稍后重跑

面试回答：403 怎么办

我不会无限重试。先检查浏览器请求和 Python 请求的差异：URL、method、headers、Cookie、Referer、Origin、params/payload、登录态。然后判断是否是权限问题、频率问题、动态参数问题或数据源限制。能合法补齐会话状态就补齐；频率问题就降速；接口参数问题就回到 Network 分析。

面试回答：429 怎么办

429 表示触发限流。我会降低并发、增加随机 sleep、使用指数退避重试、控制单 IP/账号请求频率，并记录失败 URL 后续重跑。

2.6 浏览器开发者工具 Network

Network 是你面试里最要讲得像“真实做过”的工具。面试官常问：你怎么找到接口？你怎么处理分页？你怎么知道参数变了？

Network 分析步骤

- 打开开发者工具，进入 Network。
- 勾选 Preserve log，避免跳转或刷新后请求消失。
- 筛选 Fetch/XHR，重点看接口请求。
- 刷新页面，观察初始接口。
- 执行动作：搜索、翻页、点击、滚动，观察新增请求。
- 点开请求，查看 Headers、Payload、Preview、Response。
- 复制 URL、method、params、payload、headers，用 Python 复现。
- 对比第一页和第二页参数，找 page/offset/cursor/max_id。

面试回答：如何判断数据来源

我会先看页面源码和 Elements。如果 HTML 里有目标字段，说明可以静态解析；如果源码没有数据，就看 Network 的 Fetch/XHR，请求通常返回 JSON。对于滚动或翻页页面，我会触发交互并观察新增请求，定位分页参数和停止条件。

2.7 requests 实战基础

```
import requests

session = requests.Session()
headers = {
    "User-Agent": "Mozilla/5.0",
    "Referer": "https://example.com/list",
    "Accept": "application/json, text/plain, */*",
}
params = {"page": 1, "keyword": "python"}

resp = session.get(
    "https://example.com/api/projects",
    headers=headers,
    params=params,
    timeout=10,
)
print(resp.status_code)
data = resp.json()
```

requests 重点清单

- requests.get(url, params=params)：GET 查询。
- requests.post(url, json=data)：发送 JSON 请求体。
- requests.post(url, data=data)：发送表单请求体。
- headers：模拟浏览器请求环境。
- timeout：避免请求永久卡住。
- Session：保持 Cookie 和连接复用。
- response.text：文本响应，常用于 HTML。
- response.json()：JSON 响应，常用于 API。
- response.content：二进制内容，图片、文件、PDF。

面试回答：requests.Session 有什么用

Session 可以在多个请求之间保持 Cookie 和连接状态，比如先访问首页拿到 Cookie，再请求列表页、详情页、评论接口时自动携带 Cookie。对需要会话状态的网站，Session 比每次单独 requests.get 更稳定。

2.8 HTML 解析：BeautifulSoup、XPath、CSS Selector

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(html, "html.parser")
items = soup.select(".project-card")
rows = []
for item in items:
    title_node = item.select_one(".title")
    budget_node = item.select_one(".budget")
```

```
link_node = item.select_one("a")
rows.append({
    "title": title_node.get_text(strip=True) if title_node else None,
    "budget": budget_node.get_text(strip=True) if budget_node else None,
    "url": link_node.get("href") if link_node else None,
})
```

- BeautifulSoup：适合解析普通 HTML，语法直观。
- CSS Selector：select/select_one，适合按 class/id/tag 查找。
- XPath：适合 Scrapy/lxml，表达层级结构能力强。
- 解析时要防止空标签：select_one 可能返回 None。
- 链接要处理相对路径：urljoin(base_url, href)。

面试回答：BeautifulSoup 和正则区别

BeautifulSoup 按 HTML 树结构解析，更适合提取标签、属性、链接；正则适合抽取固定格式文本，比如价格、日期、编号。复杂 HTML 不建议只靠正则，因为结构变化会让正则不稳定。

2.9 API / JSON 解析与嵌套结构

```
data = resp.json()
items = data.get("data", {}).get("items", [])
rows = []
for item in items:
    rows.append({
        "project_id": item.get("id"),
        "title": item.get("title"),
        "budget": item.get("budget"),
        "tags": ";".join(item.get("tags", [])),
        "publisher": item.get("user", {}).get("nickname"),
    })
```

JSON 清洗原则

- 先保存原始 JSON 样本，方便回溯和重跑。
- 用 get + 默认值，避免 KeyError。
- 嵌套 dict 逐层 get；嵌套 list 根据业务展开多行或拼接。
- 二级评论保留 parent_id，还原父子关系。
- 字段缺失要统计缺失率，不是随便填。
- 输出表格时字段名要稳定，不要一会中文一会英文。

面试回答：复杂嵌套 JSON 怎么办

我会先抽样观察 JSON 层级，保留原始响应。解析时使用 get 和默认值处理缺失字段；list 字段根据业务展开或拼接；父子评论保留 parent_id；最后转成 DataFrame 后检查缺失率、重复值和字段类型。

2.10 分页、滚动加载、cursor

分页类型	表现形式	处理方式
page	page=1, page=2	循环 page += 1
offset/limit	offset=0&limit=20	offset += limit
cursor	next_cursor=xxx	用上一次返回的 cursor 请求下一页

分页类型	表现形式	处理方式
max_id/min_id	社媒评论流常见	用响应返回的 max_id/min_id 继续请求
滚动加载	滚动触发接口	优先找接口；找不到再浏览器滚动

```
page = 1
while True:
    params = {"page": page, "page_size": 20}
    data = session.get(api, params=params, headers=headers, timeout=10).json()
    items = data.get("items", [])
    if not items:
        break
    save(items)
    page += 1

cursor = ""
while True:
    params = {"cursor": cursor, "limit": 20}
    data = session.get(api, params=params, headers=headers, timeout=10).json()
    items = data.get("data", [])
    save(items)
    if not data.get("has_more"):
        break
    cursor = data.get("next_cursor")
```

面试回答：如何处理分页

我会在 Network 中对比翻页或滚动时的请求，判断分页参数类型。如果是 page 就递增页码；offset/limit 就递增 offset；cursor 就用接口返回的 next_cursor；停止条件是 has_more=false、next_cursor 为空、返回列表为空或达到页数上限。

2.11 动态渲染与浏览器自动化

动态渲染指网页初始 HTML 里没有目标数据，而是页面加载后由 JavaScript 请求接口，再渲染到页面上。requests 直接请求页面时可能只拿到空壳 HTML。

工具	本质	适合场景
requests	HTTP 请求库	接口 JSON、静态 HTML
BeautifulSoup/XPath	HTML 解析	标题、正文、链接、列表项
Selenium	传统浏览器自动化	点击、输入、滚动、登录、动态页面
Playwright	现代浏览器自动化	多浏览器、异步、网络监听、上下文隔离
DrissionPage	浏览器控制 + 请求模式	动态页面、Cookie 状态复用、接口分析
CDP	Chrome DevTools Protocol	控制 Chromium 浏览器、Network、DOM、Runtime

面试回答：什么时候用浏览器自动化

如果目标数据可以在接口中稳定获取，我会优先用 requests/httpx，因为更快、更可控。只有当页面必须登录、点击、滚动、等待 JS 渲染，或者接口参数难以直接构造时，我会使用 Selenium、Playwright 或 DrissionPage。

面试回答：DrissionPage 和 CDP

DrissionPage 是 Python 网页自动化工具，CDP 是 Chrome DevTools Protocol。我的代码直接用 DrissionPage，DrissionPage 底层通过 CDP 控制 Chrome/Edge，实现打开页面、执行 JS、读取 Cookie、操作 DOM 和监听网络请求。

2.12 Scrapy 工程结构

Scrapy 不是一个请求库，而是工程化爬虫框架。你项目里有京东 Scrapy、豆瓣 Scrapy、知乎类项目，面试官可能会追问 Scrapy 五件套。

组件	作用	面试回答要点
Spider	发请求、解析响应	定义 start_urls、parse 回调、follow 下一页
Item	定义字段	让数据结构稳定，例如 title、price、url
Pipeline	清洗、去重、保存	写 CSV/Excel/DB、字段规范化、去重
Middleware	请求/响应中间层	User-Agent、代理、Cookie、异常处理
settings.py	配置中心	并发、下载延迟、Pipeline、Middleware

面试回答：Scrapy 和 requests 区别

requests 更适合简单脚本和少量接口；Scrapy 适合长期维护的结构化采集项目，有请求调度、去重、Pipeline、Middleware、并发控制和配置体系。

面试回答：Pipeline 做什么

Pipeline 接收 Spider 解析出的 Item，负责清洗、去重、格式转换和保存。比如评论数据可以在 Pipeline 中统一字段名、去重、转时间格式，并写入 CSV/数据库。

2.13 异常重试、限速、日志、断点续爬

这个岗位强调脚本维护，所以你必须懂稳定性设计。

问题	不能怎么做	正确思路
timeout	不设 timeout，一直卡住	每个请求设置 timeout
retry	无限重试	限定次数，区分错误类型
403	盲目重试	检查权限、headers、Cookie、参数、频率
429	继续加速请求	降速、退避、降低并发
日志	只 print	记录 URL、状态码、页数、失败原因
中断	从头开始	保存已采 ID，失败任务可重跑

```
import time, random, requests

def fetch_with_retry(url, headers=None, params=None, max_retries=3):
    for attempt in range(1, max_retries + 1):
        try:
            resp = requests.get(url, headers=headers, params=params, timeout=10)
            if resp.status_code == 200:
                return resp
            if resp.status_code in (500, 502, 503, 504):
                time.sleep(2 ** attempt + random.random())
```

```
        continue
    if resp.status_code == 429:
        time.sleep(10 + random.random() * 5)
        continue
    return resp
except requests.exceptions.RequestException:
    time.sleep(2 ** attempt + random.random())
return None
```

面试回答：重试机制怎么设计

我会区分错误类型。timeout、500、502、503、504 可以重试；403 要先检查权限和请求构造；429 要降速后重试；404 通常记录即可。重试次数设上限，比如 3 次，配合指数退避和失败 URL 记录。

2.14 进程、线程、协程与异步并发

概念	本质	适合场景	爬虫里怎么用
进程	操作系统资源分配单位	CPU 密集：OCR、视频转录、模型推理	图片/OCR/转录后处理
线程	进程内执行单位	I/O 密集：网络请求、文件读写	ThreadPoolExecutor + requests
协程	用户态轻量任务	大量接口请求、评论分页	asyncio + httpx/aiohttp + Semaphore

面试回答：为什么爬虫适合异步并发

爬虫大多数时间在等待网络响应，属于 I/O 密集型任务。异步并发可以在一个请求等待时切换去执行其他请求，提高吞吐量。但不能无限并发，所以要用 Semaphore 控制同时请求数量，并配合 timeout、retry 和限速。

三、第二大块：数据处理与分析基础

这个岗位不是只要你“爬下来”，还要你把数据变成可分析、可交付、可复用的结构化表。你的数据版简历里有金融文本清洗、东方财富财务风险数据采集与建模、Spark 处理大规模评分数据等经历，面试官会围绕“数据清洗、质量检查、基础分析”追问。

3.1 常见数据格式：JSON / CSV / Excel / Markdown

格式	用途	你要掌握
JSON	接口返回、嵌套数据	dict/list、get、嵌套展开、父子结构
CSV	结构化数据交换	read_csv、to_csv、encoding=utf-8-sig
Excel	业务交付	read_excel、to_excel、多 sheet、筛选透视
Markdown	流程文档	字段字典、采集流程、清洗规则、分析结论

面试回答：CSV 乱码怎么办

先判断编码，常见是 utf-8、utf-8-sig、gbk。中文 Excel 打开 CSV 乱码时，我会优先用 utf-8-sig 导出；读取时可以尝试 encoding 参数。还要注意分隔符、引号、换行符。

3.2 pandas 基础清洗函数

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.shape)
print(df.info())
print(df.isna().sum())

# 去重
df = df.drop_duplicates(subset=["project_id"])

# 缺失值
df["budget"] = df["budget"].fillna(0)

# 时间格式
df["publish_time"] = pd.to_datetime(df["publish_time"], errors="coerce")

# 文本清理
df["title"] = df["title"].astype(str).str.strip()

# 分组统计
summary = df.groupby("platform").agg(
    cnt=("project_id", "count"),
    avg_budget=("budget", "mean"),
).reset_index()

summary.to_excel("summary.xlsx", index=False)
```

必须熟练的 pandas 操作

- 读取：read_csv、read_excel。
- 查看：head、info、describe、shape、columns。
- 缺失：isna、dropna、fillna。

- 重复：duplicated、drop_duplicates。
- 类型：astype、to_datetime。
- 文本：str.strip、str.replace、str.extract、str.contains。
- 统计：groupby、agg、value_counts、pivot_table。
- 连接：merge、concat。
- 输出：to_csv、to_excel。

面试回答：你怎么做数据清洗

我会先看数据规模、字段、类型和缺失情况，比如 df.shape、df.info、df.isna().sum。然后按业务主键去重；对缺失值按字段重要性处理；对时间、金额、评分等字段做类型转换；对异常值做范围或 IQR 检查；最后输出清洗前后行数对比和字段说明。

3.3 数据质量检查

JD 特别强调“ 字段一致性、异常值和重复数据敏感 ”。这说明面试官很可能问你如何保证数据可用。

数据质检清单

- ☐ 总行数是否符合预期。
- ☐ 字段是否完整，字段名是否统一。
- ☐ 主键是否重复，例如 project_id、url、asin、comment_id。
- ☐ 缺失率是否过高。
- ☐ 字段类型是否正确：数值、时间、字符串。
- ☐ 时间格式是否统一。
- ☐ 数值范围是否合理：价格、评分、销量不能明显异常。
- ☐ 枚举值是否合法：平台来源、状态、分类。
- ☐ 采集批次是否完整。
- ☐ 清洗前后行数变化是否可解释。

面试回答：如何保障数据可用性

采集端记录状态码、页数、条数、失败 URL 和重试次数；清洗端检查字段完整性、缺失率、重复主键、异常值和类型一致性。数值字段做范围检查，时间字段转 datetime，分类字段做枚举值校验。最后输出清洗报告或 Markdown 文档说明处理规则。

3.4 缺失值、重复值、异常值

问题	常见原因	处理方式
缺失值	接口字段不存在、页面结构变化、抓取失败	核心字段缺失删除；非核心字段填充；记录缺失率
重复值	分页重叠、重复采集、增量任务未去重	按唯一 ID/URL/组合键 drop_duplicates
异常值	解析错误、单位不一致、极端数据	范围检查、IQR、分位数、业务规则
类型错误	金额含中文/符号，时间格式混乱	正则抽取、astype、to_datetime

面试回答：异常值怎么处理

先区分真实极端值和解析错误。比如预算、价格、评分等字段，要根据业务范围检查；如果是金额单位混乱，要先统一单位；如果是极端值，可以用 IQR 或分位数发现，再结合业务判断是否剔除、截尾或保留并标记。

3.5 SQL 基础

JD 只写“了解 SQL”，但你至少要会 SELECT、WHERE、GROUP BY、JOIN。

```
-- 每个平台采集条数
SELECT platform, COUNT(*) AS cnt
FROM crawl_data
GROUP BY platform
ORDER BY cnt DESC;

-- 每个平台每天采集条数
SELECT platform, DATE(created_at) AS dt, COUNT(*) AS cnt
FROM crawl_data
GROUP BY platform, DATE(created_at)
ORDER BY dt DESC;

-- 项目表关联发布方表
SELECT p.project_id, p.title, p.budget, c.company_name
FROM projects p
LEFT JOIN companies c
ON p.company_id = c.company_id;
```

pandas 与 SQL 对应

SQL	pandas
WHERE	df[condition]
GROUP BY	df.groupby()
JOIN	pd.merge()
DISTINCT	drop_duplicates()
ORDER BY	sort_values()
COUNT	count()/size()

3.6 Excel 与分析结论

- Excel 是业务交付常见格式，不只是给程序用。
- 可以按 sheet 分：原始数据、清洗后数据、统计汇总、字段说明。
- 常见操作：筛选、排序、去重、数据透视表、COUNTIF/SUMIF/XLOOKUP。
- 分析结论要写人话：不是只给表，而是告诉业务方发现了什么。

面试回答：你怎么输出结果

我会输出清洗后的 CSV/Excel 数据表，如果给业务方看会用 Excel 多 sheet 组织：清洗后明细、统计汇总、字段字典、异常记录。同时用 Markdown 写简短分析结论和采集清洗流程，方便复用。

四、第三大块：Agent / LLM 相关基础

Agent 是这个岗位的加分项，不要求你像 Agent Infra 工程师一样深，但你要能解释：爬虫和数据清洗为什么能服务 Agent。

4.1 LLM、Agent、RAG、Prompt、Evals

概念	解释	和本岗位的关系
LLM	大语言模型，擅长理解和生成文本	可以分析文本、生成代码、总结数据，但需要可靠数据输入
Agent	能拆任务、调用工具、执行步骤的 LLM 应用	需要结构化数据和可复用工具
RAG	检索增强生成：先检索，再回答	爬虫数据可进入知识库供检索
Prompt	给模型的指令和上下文	约束输出格式、字段、分析方式
Evals	评估输出质量	检查答案是否准确、是否有数据证据、是否按格式输出

面试回答：Agent 和数据采集有什么关系

Agent 要完成分析或回答，需要稳定、结构化、可解释的数据。爬虫负责从外部平台采集数据，数据清洗负责把网页/API/JSON 转成字段清楚的结构化表，这些数据才能进入 RAG、数据库或 Agent 工具调用流程。字段混乱、缺失重复多，会直接导致 Agent 产生错误分析。

4.2 AnalystGPT 项目你要怎么讲

AnalystGPT 是你非常适合这个 JD 的加分项目。你要把它讲成“CSV 数据分析 Agent”，不要讲得过度复杂。

核心链路

- 用户上传 CSV
- > FastAPI 后端读取文件
 - > pandas 解析成 DataFrame
 - > 保存 session_id
 - > 用户自然语言提问
 - > LangGraph Agent 判断调用 pandas_tool 或 matplotlib_tool
 - > 执行数据分析或生成图表
 - > 返回文本、表格、图表
 - > evidence / profiling 用于降低不可靠回答

你要掌握的词

- FastAPI：后端接口框架。
- LangGraph：Agent 流程编排和状态流转。
- pandas_tool：让 Agent 执行确定性 pandas 分析。
- matplotlib_tool：让 Agent 生成图表。
- DataFrame session：上传 CSV 后保存当前数据状态。
- ToolResult：工具返回的结构化结果。
- evidence reviewer：检查回答是否有数据证据支撑。
- session isolation：不同用户/文件的数据隔离。

面试回答：AnalystGPT 和岗位关系

AnalystGPT 让我理解了数据如何被 Agent 使用。用户上传 CSV 后，后端用 pandas 读成 DataFrame；Agent 再调用 pandas/matplotlib 工具完成统计和图表。这个过程要求底层数据字段清晰、类型稳定、缺失重复可控，所以它和岗位里的数据采集、清洗、字段结构设计非常相关。

五、结合你的项目：项目深挖与回答口径

这部分最重要。你很多项目做得时间久了，面试时容易忘细节。你不需要把每一行代码背下来，但必须记住：项目背景、数据源、字段、工具、流程、难点、结果、可优化点。

5.1 项目一：多平台数据采集与清洗

可对应：抖音、B站、豆瓣、知乎、MediaCrawler、Keep。这个项目最适合回答 JD 里的“公开网页、API、榜单、论坛、动态渲染、接口数据、JSON、CSV/Excel”。

项目复盘

维度	内容
业务目标	采集内容平台数据，形成可分析的评论、视频、字幕、用户动态、社交关系等结构化数据。
数据源	抖音评论/二级回复、B站视频元数据/字幕/封面、豆瓣/知乎内容、Keep JSON。
技术栈	requests、Scrapy、Selenium、BeautifulSoup/XPath、pandas、CSV/Excel。
关键难点	动态页面、接口分页、嵌套 JSON、二级评论父子关系、字段缺失、去重。
输出结果	评论表、二级回复表、视频元数据表、字幕文本、CSV/Excel 清洗数据。

2 分钟项目讲法

我做过多平台内容数据采集，主要包括抖音、B站、豆瓣等平台。不同平台的数据来源不一样，有的是静态 HTML，有的是接口 JSON，有的需要动态页面加载。我会先用 Network 判断数据来源，如果能找到接口就用 requests/httpx，结构化工程项目用 Scrapy，需要点击、滚动或登录时用 Selenium。采集后重点处理评论、二级回复、视频元数据、字幕等字段，通过 Pipeline 或 pandas 做去重、字段标准化和 CSV/Excel 输出。

可能追问与回答

问：二级评论怎么存？

我会把一级评论和二级回复都展开成表格行，同时保留 comment_id 和 parent_comment_id。一级评论的 parent_comment_id 为空，二级回复的 parent_comment_id 指向所属一级评论。这样既能用表格分析，也能还原评论层级。

问：B站视频数据怎么采？

先获取 BV/AV/CID 等视频标识，再调用公开视频信息接口补充标题、UP 主、发布时间、标签、简介等字段。如果需要字幕，会根据 CID 或播放器接口拿字幕地址，再下载字幕文本。最后用 pandas 合并去重。

问：这类项目如何保证数据质量？

我会用唯一 ID 或 URL 去重，检查评论内容、用户 ID、发布时间等核心字段缺失率，统一时间格式，对多平台字段做字段映射，比如 platform、content_id、user_id、content、created_at。

5.2 项目二：电商商品/评论/竞品数据采集

可对应：亚马逊、京东、单文件价格脚本。这个项目适合主打“商业数据采集与分析”。

项目复盘

维度	内容
业务目标	采集商品详情、评论、价格、评分、销量趋势和竞品数据，用于选品分析或竞品监控。
数据源	Amazon 商品页/评论页/竞品数据、京东商品与评论、价格脚本。
核心字段	ASIN/商品 ID、标题、价格、评分、评论数、评论内容、月销量、销售额、上架时间、采集时间。
技术栈	DrissionPage/CDP、requests、Scrapy、pandas、CSV/Excel、异常重试、限速。
关键难点	动态页面、会话状态、异构数据源、请求失败、去重和增量采集。

2 分钟项目讲法

我做过电商商品和竞品数据采集，核心是围绕商品 ID 或 ASIN 抽取商品标题、价格、评分、评论、销量趋势等字段。动态页面部分用 DrissionPage/CDP 控制浏览器加载页面和维护状态；可复现的接口用 requests 采集 JSON；结构化采集部分也接触过 Scrapy。采集时加入 timeout、重试、限速和增量写入，最后用 pandas 做字段清洗、去重和 CSV 落地，为选品分析提供数据。

表达边界

- 不要主动说“破解”“突破”“付费高频获取”。
- 推荐说“接口分析、动态页面处理、会话状态维护、异常重试、限速控制、结构化落地”。
- 验证码相关可说“异常场景处理/验证码识别实验”，不要把它作为核心卖点。

问：DrissionPage 和 Selenium 有什么区别？

Selenium 是传统 WebDriver 自动化，主要模拟浏览器操作；DrissionPage 更强调浏览器控制和请求模式结合，底层可通过 CDP 控制 Chromium 浏览器。对于电商页面，我可以先用浏览器加载页面、维护 Cookie，再结合接口请求批量采集。

问：电商数据怎么去重？

商品层用 ASIN/商品 ID 去重；评论层用评论 ID 或商品 ID + 用户 + 时间 + 内容 hash 去重；趋势数据用商品 ID + 日期/月度维度去重。

5.3 项目三：App / Keep 复杂 JSON 清洗

项目复盘

维度	内容
业务目标	把 App/API 返回的复杂 JSON 响应整理成用户动态、评论、二级回复等结构化表。
数据特点	深层嵌套、字段缺失、不同样本结构不一致、UGC 文本较多。
处理方法	保存原始 JSON，批量读取，get 默认值，递归解析，父子关系保留，输出 CSV/Excel。
关键能力	JSON 清洗、字段抽取、结构异构处理、数据质量检查。

2 分钟项目讲法

我处理过 App/API 的复杂 JSON 数据，原始响应里字段层级比较深，而且不同样本结构不完全一致。我会先保留原始 JSON，再写解析函数批量读取；用 get 和默认值处理字段缺失；对于嵌套 list，根据业务展开成多行；对于评论类数据，保留 parent_id 还原父子关系。最后输出 CSV/Excel，并用 pandas 检查缺失率、重复值和字段类型。

5.4 项目四：金融文本清洗与变量构造

这是你数据版简历里最强的数据清洗项目之一。它证明你不只是会爬虫，还能处理非结构化文本并构造研究变量。

项目复盘

维度	内容
数据源	沪深交易所互动平台非结构化问答记录。
数据量	约 22.8 万条金融文本样本。
处理方法	Jieba 分词、去停用词、异常文本清理、热点词典匹配。
输出	可用于回归分析的结构化文本变量。
价值	支撑论文实证检验与机制分析。

2 分钟项目讲法

我参与过金融文本数据清洗与变量构造。原始数据是沪深交易所互动平台的非结构化问答文本，规模约 22.8 万条。我用 Python 和 Jieba 做分词、去停用词、异常文本清理，再基于监管问询函和政策文本构建热点词典，通过关键词匹配识别公司回复中的热点表述，最终转成可用于回归分析的结构化文本变量。

问：文本清洗和爬虫数据清洗有什么共通点？

共通点都是把非结构化或半结构化数据转成可分析字段。爬虫数据可能来自 HTML/JSON，文本项目来自自然语言，但都要做字段标准化、异常处理、去重、缺失检查和可复现流程。文本项目还额外涉及分词、停用词、词典匹配或向量化。

5.5 项目五：新能源企业财务风险预测

这个项目非常适合回答“采集之后怎么分析”。它包括东方财富数据采集、结构化财务指标、非结构化公告文本、缺失值、异常值、特征工程、模型评估。

维度	内容
数据源	东方财富网等上市公司财务指标与公告信息。
数据类型	结构化财务数据 + 非结构化公告文本。
清洗处理	缺失值填补、IQR 异常值检测、噪声平滑。
特征工程	财务特征筛选、Doc2Vec 文本向量、SMOTE 类别平衡。
模型	Random Forest、SVM、Naive Bayes。
指标	AUC、F1 等分类指标。

2 分钟项目讲法

这个项目覆盖了数据采集、清洗、特征工程和建模。我先从东方财富采集上市公司财务指标和公告文本，构建结构化财务数据和非结构化文本融合的数据集。清洗阶段处理缺失值和异常值，用 IQR 检测极端值；文本部分用 Doc2Vec 转成语义向量；最后以 ST 企业作为风险标签，对比 Random Forest、SVM 和 Naive Bayes，使用 AUC 和 F1 评估模型。

问：IQR 是什么？

IQR 是四分位距，等于 $Q3 - Q1$ 。常用异常值规则是小于 $Q1 - 1.5 * IQR$ 或大于 $Q3 + 1.5 * IQR$ 的点作为异常值候选。它适合做数值字段的初步异常检测，但最终是否剔除还要结合业务。

5.6 项目六：AnalystGPT 数据分析 Agent

维度	内容
项目定位	CSV 对话式数据分析 Agent。
技术栈	FastAPI、LangGraph、pandas、matplotlib、前端上传。
核心链路	上传 CSV -> DataFrame session -> LLM 调用 pandas/matplotlib -> 返回文本/表格/图表。
关键价值	理解 Agent 对结构化数据、数据质量和证据校验的依赖。

2 分钟项目讲法

AnalystGPT 是一个 CSV 数据分析 Agent。用户上传 CSV 后，后端用 pandas 读成 DataFrame，并保存 session；用户用自然语言提问时，Agent 根据问题选择调用 pandas_tool 或 matplotlib_tool，执行分析代码并返回文本、表格或图表。这个项目让我理解了 Agent 对数据结构和质量的要求：字段要清楚、类型要稳定、缺失和重复要处理，否则模型生成的分析结论容易不可靠。

六、知识点 + 项目绑定表

知识点	你应该绑定的项目	一句话说法
Network 接口分析	抖音、B站、东方财富、亚马逊	先用 Network 找 XHR/API，再复现请求参数。
requests	东方财富、B站接口、普通 API	适合接口 JSON 和静态网页。
BeautifulSoup/XPath	国务院政策、豆瓣/网页文章	适合 HTML 标题、正文、链接提取。
Selenium/Playwright	动态页面、多平台采集	适合登录、点击、滚动、JS 渲染。
DrissionPage/CDP	亚马逊电商数据	浏览器控制与请求结合，处理动态页面和会话状态。
Scrapy	京东、豆瓣、知乎	Spider/Item/Pipeline/Middleware 工程化采集。
分页/cursor	抖音评论、列表页、商品评论	根据 page/offset/cursor/has_more 循环采集。
JSON 清洗	Keep、App 响应、抖音二级评论	处理嵌套、缺失、结构异构和父子关系。
pandas 清洗	金融文本、财务风险、CSV Agent	缺失、重复、异常、类型、时间、字段标准化。
SQL	结构化分析表	group by、join、统计每日/每平台数据量。
Agent/RAG	AnalystGPT	结构化数据支撑 Agent 工具调用和可信分析。

七、高频面试题回答模板

1. 给你一个网站，你怎么采？

先明确业务字段，再判断数据来源。看 HTML 是否有数据；没有就用 Network 找 XHR/API；能复现接口就用 requests/httpx；需要登录、点击、滚动或 JS 渲染就用 Selenium/Playwright/DrissionPage；长期维护用 Scrapy。采集时处理分页、headers、Cookie、timeout、retry、限速，采集后用 pandas 清洗并输出 CSV/Excel。

2. requests 和 Selenium 区别？

requests 是 HTTP 请求库，适合接口和静态网页，速度快、资源少；Selenium 是浏览器自动化，适合动态渲染、登录、点击、滚动等需要真实浏览器的场景。实际优先找接口，找不到或必须交互时再用浏览器。

3. Scrapy 和 requests 区别？

requests 更适合简单脚本；Scrapy 是工程化爬虫框架，有 Spider、Item、Pipeline、Middleware、settings、请求调度、并发和去重机制，适合多页面、多字段、长期维护项目。

4. Cookie 和 Session 区别？

Cookie 存客户端，Session 通常在服务端。服务端通过 Cookie 里的 session_id 找到会话状态。爬虫里常用 requests.Session 维持 Cookie，避免每次请求状态丢失。

5. 动态渲染是什么？

初始 HTML 里没有目标数据，页面加载后 JavaScript 请求接口并渲染数据。requests 直接拿页面可能拿不到数据，需要找接口或用浏览器自动化等待 JS 执行。

6. 如何处理分页？

在 Network 中观察翻页/滚动请求，判断是 page、offset、cursor、max_id 还是 has_more。不同分页方式用不同循环，停止条件是 has_more=false、next_cursor 为空、列表为空或页数上限。

7. 403 怎么办？

先检查请求和浏览器请求差异：URL、method、headers、Cookie、Referer、Origin、params/payload、登录态。判断是权限、频率、参数还是数据源限制。不能盲目无限重试。

8. 429 怎么办？

说明请求过快。降低并发，加入随机 sleep 或指数退避，控制单 IP/账号频率，记录失败 URL 后续重跑。

9. JSON 嵌套很深怎么办？

先保存原始 JSON，抽样观察结构。解析时用 get 和默认值；list 根据业务展开或拼接；父子评论保留 parent_id；转 DataFrame 后检查缺失、重复、类型。

10. 怎么保证数据质量？

采集端记录状态码、页数、条数、失败 URL；清洗端检查字段完整性、缺失率、重复主键、异常值、类型一致性和时间格式；输出清洗前后行数对比和字段说明。

11. pandas 怎么去重？

根据业务主键去重，例如 URL、项目 ID、ASIN、comment_id；没有唯一 ID 时可以用组合键或内容 hash。代码上用 `df.drop_duplicates(subset=[...])`。

12. 缺失值怎么处理？

先统计缺失率，再区分核心字段和非核心字段。核心字段缺失可能删除，非核心字段可填充默认值或保留为空。不能不看业务就统一填 0。

13. 异步并发是什么？

异步并发是在一个请求等待响应时切换去执行其他请求，适合大量网络 I/O。Python 中常用 `asyncio` + `httpx/aiohttp`，用 `Semaphore` 控制并发数量。

14. Agent 和数据采集关系？

Agent 需要结构化、可靠、可解释的数据作为工具调用或 RAG 的输入。爬虫采集数据，清洗后形成字段稳定的表，Agent 才能基于这些数据分析、检索和回答。

八、代码模板与排查清单

8.1 接口采集模板

```
import requests
import pandas as pd
import time, random

session = requests.Session()
headers = {
    "User-Agent": "Mozilla/5.0",
    "Accept": "application/json, text/plain, */*",
    "Referer": "https://example.com/list",
}

rows = []
for page in range(1, 51):
    params = {"page": page, "page_size": 20}
    resp = session.get("https://example.com/api/list", headers=headers, params=params, timeout=10)
    if resp.status_code == 429:
        time.sleep(10)
        continue
    if resp.status_code != 200:
        print("failed", page, resp.status_code)
        continue
    data = resp.json()
    items = data.get("items", [])
    if not items:
        break
    for item in items:
        rows.append({
            "id": item.get("id"),
            "title": item.get("title"),
            "created_at": item.get("created_at"),
        })
    time.sleep(random.uniform(1, 2))

pd.DataFrame(rows).drop_duplicates(subset=["id"]).to_csv("result.csv", index=False, encoding="utf-8-sig")
```

8.2 pandas 清洗模板

```
df = pd.read_csv("raw.csv")
raw_rows = len(df)

# 字段名统一
df.columns = [c.strip().lower() for c in df.columns]

# 去重
df = df.drop_duplicates(subset=["id"])

# 缺失检查
missing = df.isna().mean().sort_values(ascending=False)

# 时间转换
df["created_at"] = pd.to_datetime(df["created_at"], errors="coerce")

# 数值清洗
df["price"] = (
    df["price"].astype(str)
```



```
.str.replace("¥", "", regex=False)
.str.replace(",", "", regex=False)
)
df["price"] = pd.to_numeric(df["price"], errors="coerce")

# 异常值检查
q1, q3 = df["price"].quantile([0.25, 0.75])
iqr = q3 - q1
outliers = df[(df["price"] < q1 - 1.5 * iqr) | (df["price"] > q3 + 1.5 * iqr)]

clean_rows = len(df)
print("raw", raw_rows, "clean", clean_rows, "missing", missing.head())

df.to_excel("cleaned.xlsx", index=False)
```

8.3 面试前项目复盘表

项目	一定要记住的 6 点
多平台采集	数据源、字段、工具、分页/动态、清洗、输出
亚马逊电商	ASIN、商品/评论/竞品字段、DrissionPage/CDP、重试限速、ETL、选品分析
Keep/App JSON	原始 JSON、嵌套、缺失、结构异构、parent_id、CSV/Excel
金融文本	22.8 万文本、Jieba、停用词、热点词典、变量构造、论文支撑
财务风险	东方财富、财务指标、公告文本、IQR、Doc2Vec、Random Forest
AnalystGPT	CSV 上传、DataFrame session、pandas_tool、matplotlib_tool、evidence、Agent 数据质量

九、14 天冲刺复习计划

天数	主题	任务
Day 1	HTTP/URL/状态码	讲清 GET/POST、headers、Cookie、Session、403/429/500。
Day 2	Network 接口分析	找 2 个网站练 XHR、Payload、Response、分页参数。
Day 3	requests	写 GET/POST/Session/timeout/retry 模板。
Day 4	HTML 解析	练 BeautifulSoup、CSS Selector、XPath。
Day 5	JSON 与分页	练嵌套 JSON、page/offset/cursor。
Day 6	动态渲染	复习 Selenium/Playwright/DrissionPage/CDP。
Day 7	Scrapy	背熟 Spider/Item/Pipeline/Middleware/settings。
Day 8	稳定性	整理 retry、限速、日志、断点续爬回答。
Day 9	pandas 清洗	练缺失、重复、异常、时间、groupby、merge。
Day 10	SQL/Excel	练 group by、join、Excel 输出和字段字典。
Day 11	Agent	复习 Agent/RAG/Prompt/Evals/AnalystGPT。
Day 12	项目 1-3	多平台、亚马逊、Keep JSON 各练 2 分钟。
Day 13	项目 4-6	金融文本、财务风险、AnalystGPT 各练 2 分钟。
Day 14	模拟面试	按高频题录音，检查是否能讲清“采集-清洗-分析-Agent”。

十、模拟面试脚本

10.1 你为什么适合这个岗位？

我比较匹配这个岗位，因为我的经历同时覆盖外部数据采集和数据处理分析。采集侧，我做过抖音、B 站、豆瓣、亚马逊、京东、东方财富等数据源，接触过 requests、Scrapy、Selenium、DrissionPage/CDP，能根据网页结构判断是静态 HTML、接口 JSON 还是动态渲染，并处理分页、请求头、Cookie、异常重试和限速。数据侧，我做过金融文本清洗、财务风险预测和复杂 JSON 清洗，熟悉 pandas 对缺失值、重复值、异常值、时间字段和文本字段进行处理，也能输出 CSV/Excel 和分析结论。另外，我做过 AnalystGPT 这类 CSV 数据分析 Agent，理解结构化数据如何服务 LLM/Agent 场景。

10.2 你最熟的项目是什么？

如果面试官偏爬虫，优先讲“电商商品/评论/竞品数据采集”或“多平台内容数据采集”；如果偏数据分析，优先讲“新能源企业财务风险预测”或“金融文本清洗”；如果提到 Agent，优先讲 AnalystGPT。回答结构固定为：背景 -> 数据源 -> 字段 -> 技术栈 -> 难点 -> 清洗/分析 -> 输出结果 -> 反思优化。

10.3 最后反问面试官

- 这个岗位更偏日常平台数据采集，还是更偏采集后的分析报表？
- 目前数据源主要是公开网页/API，还是也会涉及需要浏览器自动化的动态页面？
- Agent 相关业务中，采集数据主要用于 RAG 知识库、评测集，还是业务分析？
- 团队对交付物更看重 CSV/Excel 表、SQL 入库，还是流程文档和可复用脚本？

十一、必须掌握的 160 个知识点清单

这一章用于查漏补缺。你不一定每个点都背定义，但至少要能用 1-2 句话解释，并知道它和哪个项目或 JD 要求相关。

A. 网络与 HTTP

- ☐ 1. URL 结构
- ☐ 2. DNS 解析
- ☐ 3. TCP 连接
- ☐ 4. HTTPS 与 TLS 基础
- ☐ 5. HTTP 请求行
- ☐ 6. HTTP 请求头
- ☐ 7. HTTP 请求体
- ☐ 8. HTTP 响应行
- ☐ 9. HTTP 响应头
- ☐ 10. 状态码分类
- ☐ 11. GET/POST/PUT/DELETE
- ☐ 12. Query Params
- ☐ 13. Payload
- ☐ 14. Content-Type
- ☐ 15. Accept
- ☐ 16. User-Agent
- ☐ 17. Cookie
- ☐ 18. Session
- ☐ 19. Token
- ☐ 20. Referer
- ☐ 21. Origin
- ☐ 22. 重定向
- ☐ 23. 编码与乱码
- ☐ 24. HTML/JSON/图片/PDF 响应
- ☐ 25. Keep-Alive
- ☐ 26. 代理服务器
- ☐ 27. 抓包工具
- ☐ 28. CORS 基础
- ☐ 29. CSRF Token 基础
- ☐ 30. WebSocket 了解

B. 网页与接口分析

- ☐ 1. Elements 面板
- ☐ 2. Network 面板
- ☐ 3. Fetch/XHR 筛选
- ☐ 4. Headers
- ☐ 5. Payload

- ☐ 6. Preview
- ☐ 7. Response
- ☐ 8. Application 面板
- ☐ 9. Cookie 查看
- ☐ 10. LocalStorage 查看
- ☐ 11. 接口复现
- ☐ 12. 接口参数对比
- ☐ 13. 翻页参数分析
- ☐ 14. 滚动加载分析
- ☐ 15. 登录态判断
- ☐ 16. 静态 HTML 判断
- ☐ 17. 动态渲染判断
- ☐ 18. API JSON 判断
- ☐ 19. GraphQL 了解
- ☐ 20. 移动端接口了解
- ☐ 21. 接口返回字段映射

C. Python 爬虫工具

- ☐ 1. requests.get
- ☐ 2. requests.post
- ☐ 3. headers 参数
- ☐ 4. params 参数
- ☐ 5. data 参数
- ☐ 6. json 参数
- ☐ 7. timeout
- ☐ 8. proxies
- ☐ 9. requests.Session
- ☐ 10. response.text
- ☐ 11. response.json
- ☐ 12. BeautifulSoup
- ☐ 13. CSS Selector
- ☐ 14. XPath
- ☐ 15. lxml
- ☐ 16. 正则辅助提取
- ☐ 17. Selenium
- ☐ 18. Playwright
- ☐ 19. DrissionPage
- ☐ 20. CDP
- ☐ 21. Scrapy
- ☐ 22. httpx
- ☐ 23. aiohttp
- ☐ 24. asyncio
- ☐ 25. logging

D. 采集工程稳定性

- ☐ 1. 分页 page
- ☐ 2. offset/limit
- ☐ 3. cursor
- ☐ 4. max_id/min_id
- ☐ 5. has_more
- ☐ 6. 断点续爬
- ☐ 7. 增量采集
- ☐ 8. 失败 URL 记录
- ☐ 9. 重试上限
- ☐ 10. 指数退避
- ☐ 11. 随机 sleep
- ☐ 12. 限速
- ☐ 13. 并发控制
- ☐ 14. Semaphore
- ☐ 15. 线程池
- ☐ 16. 协程
- ☐ 17. 进程
- ☐ 18. 任务队列概念
- ☐ 19. 日志字段
- ☐ 20. 数据落盘策略
- ☐ 21. 原始数据保留
- ☐ 22. 字段字典
- ☐ 23. 采集批次 ID

E. 反爬与合规表达

- ☐ 1. User-Agent 检测
- ☐ 2. Referer 检测
- ☐ 3. Cookie 检测
- ☐ 4. 频率限制
- ☐ 5. 验证码了解
- ☐ 6. 登录态限制
- ☐ 7. 动态参数
- ☐ 8. 接口签名了解
- ☐ 9. 浏览器自动化特征
- ☐ 10. robots/服务条款意识
- ☐ 11. 公开数据优先
- ☐ 12. 最小化采集
- ☐ 13. 不采敏感信息
- ☐ 14. 不展示 token/cookie
- ☐ 15. 避免“破解/绕过”表述
- ☐ 16. 用“接口分析/异常处理/会话维护”表述

F. 数据格式与清洗

- ☐ 1. JSON 嵌套
- ☐ 2. list/dict 区分
- ☐ 3. 字段缺失
- ☐ 4. 结构异构
- ☐ 5. 父子评论关系
- ☐ 6. CSV 编码
- ☐ 7. Excel 多 sheet
- ☐ 8. Markdown 文档
- ☐ 9. pandas DataFrame
- ☐ 10. read_csv
- ☐ 11. read_excel
- ☐ 12. info
- ☐ 13. describe
- ☐ 14. isna
- ☐ 15. dropna
- ☐ 16. fillna
- ☐ 17. duplicated
- ☐ 18. drop_duplicates
- ☐ 19. astype
- ☐ 20. to_datetime
- ☐ 21. str.extract
- ☐ 22. str.replace
- ☐ 23. groupby
- ☐ 24. agg
- ☐ 25. value_counts
- ☐ 26. merge
- ☐ 27. concat
- ☐ 28. pivot_table
- ☐ 29. to_csv
- ☐ 30. to_excel

G. 数据质量与分析

- ☐ 1. 主键设计
- ☐ 2. 唯一约束
- ☐ 3. 重复率
- ☐ 4. 缺失率
- ☐ 5. 异常值
- ☐ 6. IQR
- ☐ 7. Z-score 了解
- ☐ 8. 单位标准化
- ☐ 9. 时间标准化
- ☐ 10. 枚举值校验

- ☐ 11. 字段一致性
- ☐ 12. 采集完整性
- ☐ 13. 数据血缘
- ☐ 14. raw/staging/clean 分层
- ☐ 15. group by
- ☐ 16. join
- ☐ 17. left join
- ☐ 18. count distinct
- ☐ 19. SQL where
- ☐ 20. SQL order by
- ☐ 21. Excel 透视表
- ☐ 22. COUNTIF/SUMIF
- ☐ 23. 分析结论
- ☐ 24. 可复现性

H. Agent 相关

- ☐ 1. LLM
- ☐ 2. Agent
- ☐ 3. Tool Calling
- ☐ 4. RAG
- ☐ 5. Embedding 了解
- ☐ 6. Prompt
- ☐ 7. System Prompt
- ☐ 8. Evals
- ☐ 9. Hallucination
- ☐ 10. Evidence
- ☐ 11. ToolResult
- ☐ 12. DataFrame session
- ☐ 13. FastAPI
- ☐ 14. LangGraph
- ☐ 15. pandas_tool
- ☐ 16. matplotlib_tool
- ☐ 17. session isolation
- ☐ 18. 数据画像
- ☐ 19. 缺失值检查工具
- ☐ 20. 图表生成
- ☐ 21. Markdown 报告

十二、知识点详细补充：面试官爱深挖的地方

12.1 HTTP 报文到底是什么

HTTP 请求不是一句“访问 URL”那么简单，它由请求行、请求头、空行、请求体组成。响应由状态行、响应头、空行、响应体组成。爬虫复现接口时，本质上是在复现浏览器发出的 HTTP 请求。

```
GET /api/list?page=1 HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Accept: application/json
Cookie: session_id=xxx
```

GET 通常没有请求体

```
HTTP/1.1 200 OK
Content-Type: application/json
Set-Cookie: session_id=yyy
```

```
{"items": [...], "has_more": true}
```

深挖回答

如果面试官问“你复现接口到底复现了什么”，你要说：复现 method、URL、query params、headers、cookie、payload 和必要的会话状态；响应回来后根据 Content-Type 判断解析方式。

12.2 Content-Type、data、json、form-data

Content-Type	Python 写法	常见场景
application/json	requests.post(url, json=obj)	接口提交 JSON
application/x-www-form-urlencoded	requests.post(url, data=dict)	普通表单提交
multipart/form-data	requests.post(url, files=files)	文件上传
text/html	response.text + BeautifulSoup	网页 HTML
application/json 响应	response.json()	接口返回 JSON

深挖回答

POST 请求失败时，我会检查 Content-Type 和 Payload 是否匹配。浏览器里如果是 JSON payload，Python 就应该用 json=；如果是 Form Data，就用 data=；文件上传则用 files=。

12.3 编码、乱码、二进制响应

- HTML 文本乱码：检查 response.encoding、apparent_encoding、页面 meta charset。
- CSV 中文乱码：导出给 Excel 时常用 utf-8-sig；国内老数据可能是 gbk。
- JSON 一般是 UTF-8，但也可能包含转义字符。
- 图片、PDF、视频等用 response.content，不要用 response.text。
- 保存二进制时用 wb 模式。

```
# 保存图片/PDF
resp = requests.get(file_url, timeout=10)
with open("file.bin", "wb") as f:
    f.write(resp.content)
```

12.4 CORS、CSRF、Token：知道到什么程度

CORS 是浏览器的跨域安全策略，主要限制浏览器脚本，不直接限制 Python requests。CSRF Token 是网站为了防止跨站请求伪造设置的校验参数，可能出现在表单、Cookie、meta 标签或接口返回里。Token 则常用于认证授权。

面试口径

如果请求里有 csrf_token 或 authorization，我会先判断它是否来自合法登录流程或公开接口返回，不会伪造敏感凭证。技术上需要在 Network 或页面里找到 token 来源，并在合法授权范围内复现请求。

12.5 反爬知识怎么讲才安全

这个岗位不是“逆向攻防岗”。你可以展示你理解反爬，但表达要偏稳定性和合规采集，不要强调突破。

不要这样说	换成这样说
破解签名	分析接口参数与请求依赖
绕过风控	处理异常访问限制与请求稳定性
突破验证码	处理验证码等异常交互场景
高频获取付费数据	在授权范围内进行批量数据采集与限速控制
中间人解密	使用抓包工具分析自己授权设备的请求结构

12.6 Scrapy 请求生命周期

Scrapy 面试深挖时，常问“一个请求在 Scrapy 里怎么流转”。

- Spider 产生 Request
- > Scheduler 调度请求
- > Downloader Middleware 处理请求，如 UA/代理/Cookie
- > Downloader 发起网络请求
- > Response 返回
- > Spider Middleware 处理响应
- > Spider parse 解析出 Item 或新的 Request
- > Item 进入 Pipeline 清洗/去重/保存
- > 新 Request 继续进入 Scheduler

深挖回答

Scrapy 的优势是把请求调度、并发、去重、下载、解析、数据保存拆成模块。项目变复杂时，比单个 requests 脚本更容易维护。

12.7 Playwright / Selenium / DrissionPage 深挖

工具	核心对象/概念	你要会说
Selenium	WebDriver、WebElement、WebDriverWait	传统浏览器自动化，适合点击、输入、等待。
Playwright	Browser、Context、Page、Locator	现代自动化，支持上下文隔离、网络监听、异步。
DrissionPage	浏览器模式 + 请求模式	适合动态页面与请求结合，底层可用 CDP 控制浏览器。
CDP	Page、Network、Runtime、DOM domain	Chrome 开发者工具协议，可控制页面和监听网络。

深挖回答：等待元素

动态页面不能一打开就取元素，要等待目标元素出现或接口返回。Selenium 常用 WebDriverWait，Playwright 常用 locator.wait_for 或自动等待。这样比固定 sleep 更稳定。

12.8 并发设计：不是越快越好

面试官问并发，通常不是只想听“提高速度”，还想听你如何控制稳定性。

- 网络 I/O 密集：线程或协程都能提升吞吐。
- CPU 密集：OCR、视频转录、模型推理用多进程更合理。
- 并发要配合限速，避免 429 或影响目标服务。
- 并发任务要有失败重试和失败记录。
- 不同域名、不同账号、不同接口可以设置不同限速策略。

深挖回答

我不会盲目把并发开很大。会先测单请求稳定性，再逐步增加并发，观察状态码、响应时间、失败率。如果出现 429 或失败率上升，就降低并发或增加退避。

12.9 数据分层：raw / staging / clean

数据处理面试里，如果你能讲 raw/staging/clean，会显得更像真实做过数据工程。

层级	含义	作用
raw	原始响应或原始表	保留证据，方便回溯和重跑
staging	初步解析后的中间表	字段初步展开，但可能未完全清洗
clean	清洗后的可分析表	字段稳定、去重、类型统一、可交付

深挖回答

对于复杂采集任务，我倾向于保留原始 JSON/HTML 或原始 CSV，再生成清洗表。这样如果解析规则变了，可以从 raw 层重跑，不需要重新采集。

12.10 字段字典怎么写

字段	类型	说明	来源	清洗规则
project_id	string	项目唯一 ID	接口 id 字段	去重主键
title	string	项目标题	HTML/API title	strip 去空格
budget	float	预算金额	文本/JSON	去符号、统一单位
publish_time	datetime	发布时间	页面/接口	to_datetime
platform	string	平台来源	采集配置	固定枚举

深挖回答

字段字典能保证后续分析和 Agent 使用时理解一致。里面至少包括字段名、类型、含义、来源、是否必填、清洗规则和示例。

十三、项目经理深挖题库：逐项目准备

这一章模拟“项目经理/面试官”深挖项目。每个项目你都要准备：背景、字段、技术、难点、质量保障、结果、复盘。

多平台数据采集

- ☐ 1. 你为什么选 requests/Scrapy/Selenium ?
- ☐ 2. 抖音和 B 站的数据结构有什么不同 ?
- ☐ 3. 二级评论怎么保留父子关系 ?
- ☐ 4. 字幕数据如何和视频元数据关联 ?
- ☐ 5. Pipeline 做了哪些清洗 ?
- ☐ 6. 怎么判断重复评论 ?
- ☐ 7. 动态页面如何等待 ?
- ☐ 8. 如果接口字段变了怎么办 ?
- ☐ 9. 数据最后怎么交付 ?
- ☐ 10. 如果重做, 你会怎么工程化 ?

准备方法：每个问题至少准备 3 句话：先说结论，再说项目里怎么做，最后说如果优化会怎么做。

亚马逊/电商数据采集

- ☐ 1. ASIN 是什么，为什么适合作主键？
- ☐ 2. 商品表、评论表、趋势表怎么设计？
- ☐ 3. DrissionPage/CDP 解决了什么问题？
- ☐ 4. 哪些数据用浏览器，哪些数据用接口？
- ☐ 5. 如何做增量采集？
- ☐ 6. 评论如何去重？
- ☐ 7. 价格和销量字段怎么标准化？
- ☐ 8. 请求失败怎么处理？
- ☐ 9. 如何避免过高频率？
- ☐ 10. 如何把数据用于选品分析？

准备方法：每个问题至少准备 3 句话：先说结论，再说项目里怎么做，最后说如果优化会怎么做。

App/Keep JSON 清洗

- ☐ 1. 原始 JSON 为什么要保留？
- ☐ 2. 字段缺失怎么处理？
- ☐ 3. 结构异构是什么意思？
- ☐ 4. 递归解析怎么设计？
- ☐ 5. list 字段展开成多行还是拼接？
- ☐ 6. 二级回复怎么建表？
- ☐ 7. 如何检查解析是否漏字段？
- ☐ 8. 清洗后怎么验证？
- ☐ 9. CSV/Excel 如何输出？
- ☐ 10. 这个项目体现了什么数据能力？

准备方法：每个问题至少准备 3 句话：先说结论，再说项目里怎么做，最后说如果优化会怎么做。

金融文本清洗

- ☐ 1. 为什么要分词？
- ☐ 2. 为什么要去停用词？
- ☐ 3. 热点词典如何构建？
- ☐ 4. 关键词匹配有什么局限？

- ☐ 5. 异常文本有哪些？
- ☐ 6. 如何保证变量可复现？
- ☐ 7. 文本变量怎么进入回归？
- ☐ 8. 和爬虫清洗有什么共通？
- ☐ 9. 如果换成 LLM 标注会怎么做？
- ☐ 10. 这个项目和岗位的数据处理有什么关系？

准备方法：每个问题至少准备 3 句话：先说结论，再说项目里怎么做，最后说如果优化会怎么做。

新能源财务风险预测

- ☐ 1. 为什么选东方财富？
- ☐ 2. 结构化和非结构化数据怎么融合？
- ☐ 3. IQR 异常值如何处理？
- ☐ 4. Doc2Vec 是什么？
- ☐ 5. SMOTE 为什么用？
- ☐ 6. 为什么 Random Forest 表现好？
- ☐ 7. AUC 和 F1 怎么解释？
- ☐ 8. 特征工程有哪些？
- ☐ 9. 如果不建模，只做分析会输出什么？
- ☐ 10. 这个项目如何体现“采集+分析”闭环？

准备方法：每个问题至少准备 3 句话：先说结论，再说项目里怎么做，最后说如果优化会怎么做。

AnalystGPT

- ☐ 1. CSV 上传后数据如何保存？
- ☐ 2. DataFrame session 是什么？
- ☐ 3. LLM 怎么调用 pandas？
- ☐ 4. matplotlib_tool 做什么？
- ☐ 5. 如何避免模型乱编分析？
- ☐ 6. evidence reviewer 是什么？
- ☐ 7. session isolation 为什么重要？
- ☐ 8. 如果 CSV 有脏数据怎么办？
- ☐ 9. 和 RAG 有什么区别？
- ☐ 10. 这个项目如何服务本岗位 Agent 需求？

准备方法：每个问题至少准备 3 句话：先说结论，再说项目里怎么做，最后说如果优化会怎么做。

十四、临场回答公式

14.1 项目回答公式：B-D-T-D-R-O

字母	含义	你要说什么
B	Background 背景	为什么做这个项目，业务目标是什么
D	Data 数据	数据源、数据量、字段、数据格式
T	Tech 技术	用了什么工具，为什么这么选

字母	含义	你要说什么
D	Difficulty 难点	分页、动态、JSON、缺失、异常、并发等
R	Result 结果	输出表、分析结论、模型指标、业务价值
O	Optimization 优化	如果重做，如何工程化、监控、文档化

14.2 排查回答公式：现象 -> 定位 -> 处理 -> 预防

比如 403：现象是接口返回 403；定位时对比浏览器和 Python 请求的 URL、headers、Cookie、Referer、params；处理时补齐合法请求信息或降频；预防时增加日志、限速、失败重跑和字段监控。

14.3 数据质量回答公式：完整性 -> 唯一性 -> 合法性 -> 一致性 -> 可追溯

- 完整性：字段是否缺失、采集页数是否完整。
- 唯一性：主键是否重复。
- 合法性：数值范围、时间格式、枚举值是否合法。
- 一致性：多来源字段名、单位、类型是否统一。
- 可追溯：保留原始数据、清洗规则、行数变化。

十五、最终速记卡：面试前 30 分钟看这一页

15.1 你的人设

我是偏数据采集与数据处理分析的候选人。能拆外部数据源，分析网页/API/动态渲染，用 Python 爬虫稳定采集，再用 pandas/SQL/Excel 清洗、质检和分析，并理解数据如何服务 Agent/RAG。

15.2 必背 8 句话

- 优先找接口，找不到或必须交互时再用浏览器自动化。
- 403 不盲目重试，先检查 headers、Cookie、Referer、Origin、权限和参数。
- 429 是限流，要降并发、随机 sleep、指数退避和失败记录。
- 分页先用 Network 对比参数，常见 page、offset、cursor、max_id、has_more。
- JSON 清洗要保留原始数据，用 get 处理缺失，父子评论保留 parent_id。
- 数据质量检查看字段完整性、重复主键、缺失率、异常值、类型和时间格式。
- Scrapy 的核心是 Spider、Item、Pipeline、Middleware、settings。
- Agent 需要结构化、稳定、可解释的数据，否则容易产生错误分析。